

## CMPS 102 — Spring 2020 – Homework 3

Updated Ver. 1(25-Oct)

This assignment comprises of five questions and is worth fifty points. It is due on November 09 (10 a.m.) on Gradescope.

Before you begin the assignment, please read the following carefully.

- Read the *Homework Guidelines*.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- You are **not** allowed to work in teams.
- A subset of the questions may be graded due to limited resources.

*I have read and agree to the collaboration policy.* – Baladithya Balamurugan, bbalamur@ucsc.edu  
Collaborators:

- 
1. (Total: 5 points) Prove that if  $h_1(n) = \Theta(f(n))$  and  $h_2(n) = \Theta(g(n))$ , then  $h_1(n)h_2(n) = \Theta(f(n)g(n))$ .

**Solution.**

Since  $h_1(n) = \Theta(f(n))$  then there exists constants  $c_1, c_2$ , and  $\forall n > N1$  such that:

$$0 \leq c_1 \cdot f(n) \leq h_1(n) \leq c_2 \cdot f(n)$$

and  $h_2(n) = \Theta(g(n))$  there exists constants  $c_3, c_4$ , and  $\forall n > N2$  such that:

$$0 \leq c_3 \cdot g(n) \leq h_2(n) \leq c_4 \cdot g(n)$$

These are the definitions for  $h_1$  and  $h_2$  respectively and if we take a look at

$$h_1(n) \cdot h_2(n) \quad \forall n > \max N1, N2$$

$$(c_1 \cdot f(n)) \cdot (c_3 \cdot g(n)) \leq h_1(n) \cdot h_2(n) \leq (c_2 \cdot f(n)) \cdot (c_4 \cdot g(n))$$

$$(c_1 \cdot c_3 \cdot f(n) \cdot g(n)) \leq h_1(n) \cdot h_2(n) \leq (c_2 \cdot c_4 \cdot f(n) \cdot g(n))$$

Since  $c_1, c_2, c_3$ , and  $c_4$  are all constants we can combine  $c_1$  and  $c_3$  into a new constant  $c_5$  and  $c_2$  and  $c_4$  into  $c_6$ :

$$(c_5 \cdot f(n) \cdot g(n)) \leq h_1(n) \cdot h_2(n) \leq (c_6 \cdot f(n) \cdot g(n))$$

This format is the Asymptotic definition of Theta therefore:

$$h_1(n) \cdot h_2(n) = \Theta(f(n)g(n)) \quad \exists c_5, c_6 | \forall n > \max(N1, N2)$$

2. (Total: 15 Points) The following recursive algorithm determines whether an array is sorted. Variables  $B_1, B_2$  and  $B_3$  are Boolean, and  $\wedge$  represents the Logical And operator.

```
1: function FOO( $A, p, r$ )                                ▷ takes as input an array  $A[p..r]$ ; precondition  $r \geq p$ 
2:   if  $r=p$  then
3:     return TRUE
4:   else
5:      $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
6:      $B_1 \leftarrow \text{FOO}(A, p, q)$ 
7:      $B_2 \leftarrow \text{FOO}(A, q+1, r)$ 
8:      $B_3 \leftarrow (A[q] \leq A[q+1])$ 
9:     return  $(B_1 \wedge B_2 \wedge B_3)$ 
10:  end if
11: end function
```

- (a) (1 points) Describe precisely, in a few sentences, what the above algorithm, FOO, does.

**Solution.** *The algorithm checks if the array given between indices  $p$  and  $r$  is sorted. The algorithm uses a mergesort type approach and splits the array/subarray in half until the base case of just 1 array element is reached. Once the base case is reached then the algorithm returns true and checks if middle index of the array is less than or equal than the one to the direct right of the middle element. Through that there is a check for the order of elements at the base level of a sub array of size 2. As long as there is 2 elements  $B_1$  and  $B_2$  will return true just for having a single variable in each half of the array and the variable  $B_3$  checks the order and if all 3 variables register as true then the array/subarray is sorted.*

(b) (7 points) Prove, using induction, that the algorithm does what you claimed in (a).

**Solution.** Consider an array of size 1. The array is already sorted since there are no other elements to compare against. Therefore the base case is correct since it returns the boolean true for an array size 1.

Then for array sizes greater than 1 we use the induction step. Since we know that the algorithm correctly returns true for a size 1 array then we suppose that the algorithm correctly determines an array of size  $n$  is correctly sorted. The algorithm will recursively call Foo on the 2 sub arrays of size  $n/2$  if Foo is called on an array of size  $n$ . By induction hypothesis the calls will correctly return true since the recursive calls keep splitting the subarrays down to an array size 1 which is the base case. Then if the middle value and its right adjacent value is sorted correctly then the array is sorted. The reason for the middle and right adjacent value is because they are the ending and beginning of the sub arrays respectively.

- (c) (2 points) Let  $T(n)$  be the number of array comparisons (discussed in lecture) the algorithm does on an array of size  $n$ . Write a recurrence relation of  $T(n)$ .

**Solution.**

$$T(n) = \begin{cases} 0 & n = 1 \\ 2 \cdot T(\frac{n}{2}) + 1 & n > 1 \end{cases}$$

*Each call of  $\text{Foo}(n)$  if greater than 1 will result in 2 Recursive calls of  $\text{Foo}(\frac{n}{2})$  and a single array comparison of the middle and its right adjacent elements.*

(d) (2 points) Use Masters Theorem to find a tight bound for  $T(n)$ .

**Solution. MT:**

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 1$$

In the form  $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$

$$a = 2$$

$$b = 2$$

$$f(n) = 1 = n^0$$

Now we compare  $n^{\log_b a}$  and  $n^0$

$$\log_b a = \log_2 2 = 1$$

$$n^{\log_b a} \approx n^{\log_2 2} \text{ vs } n^0 \Rightarrow n^1 > n^0$$

since

$$n^{\log_b a} > n^0$$

we can move forward with Case 1:

"Look" for  $\epsilon$ :

$$\log_2 2 - \epsilon \geq 0, \quad \epsilon > 0$$

$$-\epsilon \geq -\log_2 2$$

$$\epsilon \leq \log_2 2$$

so

$$0 < \epsilon \leq 1$$

$$O(n^0) = O(n^{\log_2 2 - \epsilon})$$

If we take  $\epsilon$  to be 1 then:

$$O(n^0) = O(n^{\log_2 2 - 1}) \Rightarrow O(n^0)$$

$$f(n) = O(n^0) \equiv O(n^{\log_2 2 - 1}) \Rightarrow O(n^0)$$

Therefore

$$f(n) = O(n^{\log_b a - \epsilon})$$

By Case 1:

$$T(n) = \Theta(n^{\log_2 2}) = \Theta(n^1) = \Theta(n)$$

(e) (3 points) Obtain an exact solution of  $T(n)$  and verify the bound.

**Solution.**

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 1$$

$$T(n) = 2 \cdot (2 \cdot T\left(\frac{n}{4}\right) + 1) + 1$$

$$T(n) = 2 \cdot (2 \cdot (2 \cdot T\left(\frac{n}{8}\right) + 1) + 1) + 1$$

$$T(n) = 2 \cdot (2 \cdot (2 \cdot (2 \cdot T\left(\frac{n}{16}\right) + 1) + 1) + 1) + 1$$

$$T(n) = 2^k \cdot T\left(\frac{n}{2^k}\right) + (2^k - 1)$$

Assume  $\frac{n}{2^k} = 1$  for the base case

$$\frac{2^k}{n} = 1$$

$$2^k = n$$

$$k = \log_2 n$$

$$T\left(\frac{n}{2^k} = 1\right) = 0$$

$$T(n) = 2^{\log_2 n} \cdot 0 + (2^{\log_2 n} - 1)$$

$$T(n) = (2^{\log_2 n} - 1)$$

$$T(n) = (n - 1) \Rightarrow \Theta(n)$$

3. (Total: 5 pts) Rank the following functions based on their asymptotic growth rate with the SLOWEST growing function on the TOP and the FASTEST-growing function on the BOTTOM.

If some functions have the same asymptotic growth rate (i.e. they are  $\Theta()$  of each other), then put the tied functions on the same line separated by '='. Otherwise, each line should have exactly one function.

- (a)  $n \log n$
- (b)  $n$
- (c)  $\log(\log n)$
- (d)  $n^2$
- (e)  $2^n$
- (f)  $2^{2^n}$
- (g)  $2^{\log n}$
- (h)  $n^{\frac{1}{2}}$
- (i)  $n^{2000}$
- (j)  $6^n$
- (k)  $\log(n^3)$
- (l)  $n^{\log 4}$

**Solution.**

- (a)  $\log(\log n)$
- (b)  $\log(n^3)$
- (c)  $n^{\frac{1}{2}}$
- (d)  $2^{\log n} = n$
- (e)  $n \log n$
- (f)  $n^{\log 4} = n^2$
- (g)  $n^{2000}$
- (h)  $2^n$
- (i)  $6^n$
- (j)  $2^{2^n}$

4. (Total: 5 points) Use the  $c$  and  $n_0$  style definition to *prove* that that  $n^3$  is *not* in  $O(n^2)$ .

**Solution.**

***Proof by Contradiction***

*Asymptotic Definition of big-O:*

$$f(n) = O(g(n))$$

*iff*

$$O(g(n)) = f(n) | \exists c > 0 \ \&\& \ n_0 > 0$$

$$0 \leq f(n) \leq c \cdot g(n)$$

$$\forall n \geq n_0$$

*Assume that  $n^3 = O(n^2)$ . Then:*

$$\forall n \geq n_0 \quad \&\& \quad c > 0$$

$$0 \leq n^3 \leq c \cdot n^2$$

$$0 \leq 1 \leq \frac{c \cdot n^2}{n^3}$$

$$0 \leq 1 \leq c \cdot \frac{n^2}{n^3}$$

$$0 \leq 1 \leq c \cdot \frac{1}{n}$$

$$0 \leq n \leq c$$

*For all  $n \leq c$ ,  $n^3 \geq n^2$ . Since the max  $c$  can be is  $n$ , the inequality  $n^3 \geq n^2$  holds. From this we see that what we found is a contradiction to the Asymptotic Definition of big-O (line 2 after the assumption) therefore  $n^3$  is not in  $O(n^2)$ .*



5. (Total: 8 points) Determine the largest integer  $k$  such that if there is a way to multiply  $3 \times 3$  matrices using  $k$  multiplications (not assuming commutativity of multiplication of matrix elements), then there is an algorithm to multiply  $n \times n$  matrices (where  $n$  is an exact power of 3) in time  $O(n^{\log 7})$ . State your reasoning clearly. What would the run time of this algorithm be? (Hint: Proceed as in the analysis of Strassen's algorithm, but recur on submatrices of size  $\frac{n}{3} \times \frac{n}{3}$ .)

**Solution.**

Since we are using  $3^n \times 3^n$  matrices the recurrence for the modified version of Strassen's algorithm is as follows:

$$T(n) = k \cdot T\left(\frac{n}{3}\right) + O(n^2)$$

In the recurrence there are  $k$  recursive calls with each call on a third of the matrix. Then there is  $O(n^2)$  work in each recursive call. Given the aforementioned information and the format for a default Master's Theorem equation ( $T(n) = a \cdot T(\frac{n}{b}) + f(n)$ ): MT:

$$T(n) = k \cdot T\left(\frac{n}{3}\right) + n^2$$

In the form  $T(n) = a \cdot T(\frac{n}{b}) + f(n)$

$$a = k \quad b = 3 \quad f(n) = n^2$$

Now we compare  $n^{\log_b a}$  and  $n^2$

$$\log_b a = \log_3 k = 2$$

For now let  $k = 10$ , since the algorithm for  $2 \times 2$  used a  $k = 7$  so we can safely assume that  $k$  is a number greater than 7.

$$n^{\log_b a} \approx n^{\log_3 k} \text{ vs } n^2 \Rightarrow n^{\log_3 10} > n^2 \Rightarrow n^{2.1} > n^2$$

since

$$n^{\log_b a} > n^0$$

we can move forward with Case 1: Therefore

$$f(n) = O(n^{\log_b a - \epsilon})$$

By Case 1:

$$T(n) = \Theta(n^{\log_3 k})$$

Since we are given a bound of  $O(n^{\log 7})$  we can check the validity:

$$\Theta(n^{\log_3 k}) \leq \Theta(n^{\log 7})$$

$$n^{\log_3 k} \leq n^{\log 7} \Rightarrow \log_3 k \leq \log 7$$

$$\frac{\log k}{\log 3} \leq \frac{\log 7}{\log 2} \Rightarrow \log k \leq \frac{\log 7}{\log 2} \cdot \log 3$$

$$\log k \leq \frac{4}{3}$$

$$k \leq 10^{\frac{4}{3}} = 10 \cdot 10^{(1/3)} \approx 21.5444 \approx 22$$

$$k \leq 22$$

This means that there are at max 22 (if we round) multiplication operations for the Strassen algorithm that runs on  $3^n \times 3^n$  matrices. And from this we can finally calculate the runtime from before:

$$\Theta(n^{\log_3 k}) = \Theta(n^{\log_3 22}) \approx \Theta(n^{2.813588}) \approx \Theta(n^{2.81})$$

6. (Total: 12 points) In lecture, you saw Karatsuba's algorithm for integers in base 2 (binary).

- (a) (5 points) Now modify and write the algorithm `Karatsuba(X, Y)` for integers in base 10. Write a recurrence and compare it with the one we obtained in lecture.

---

**Algorithm 1** Karatsuba - Vishal ed.

---

```
1: function KARATSUBA(x,y)
2:   n ← max[len(abs(x)), len(abs(y))]
3:   if n == 1 then
4:     return x · y
5:   end if
6:   m ←  $\frac{n}{2}$ 
7:   x1 ← ⌊ x / 10**m ⌋
8:   x2 ← x % 10**m
9:   y1 ← ⌊ y / 10**m ⌋
10:  y2 ← y % 10**m
11:  u ← karatsuba(x1,y1)
12:  v ← karatsuba(x2,y2)
13:  w ← karatsuba(x1-x2,y1-y2)
14:  z ← u+v-w
15:  return u · (10n) + z · (10m) + d
16: end function
```

---

**Solution.**

- (b) (2 points) If the input to the algorithm are 1042 and 4200, i.e., Karatsuba (1042, 4200) list the sub-calls and the corresponding values of the parameters X and Y.

**Solution.**

**Format:** *Call Level # x y*

*Call Level 0 1042 4200*

*Call Level 1 10 42*

*Call Level 2 1 4*

*Call Level 2 0 2*

*Call Level 2 1 2*

*Call Level 1 42 0*

*Call Level 2 4 0*

*Call Level 2 2 0*

*Call Level 2 2 0*

*Call Level 1 -32 42*

*Call Level 2 -4 4*

*Call Level 2 8 2*

*Call Level 2 -12 2*

*Call Level 3 -2 0*

*Call Level 3 8 2*

*Call Level 3 -10 -2*

*Call Level 4 -1 -1*

*Call Level 4 0 8*

*Call Level 4 -1 -9*

- (c) (5 points) Find the product of 1042 and 4200 using the Karatsuba (1042, 4200) algorithm. Clearly state the algebra and result for every subcall.

**Solution. Format: Call Level # x y**

**Call Level 0 1042 4200**

$n = \max \text{ length of } x \text{ or } y = 4$   
 $m = n/2 = 2$   
 $x1 = 10$   
 $x2 = 42$   
 $y1 = 42$   
 $y2 = 00$

**Call Level 1 10 42**

$n = \max \text{ length of } x \text{ or } y = 2$   
 $m = n/2 = 1$   
 $x1 = 1$   
 $x2 = 0$   
 $y1 = 4$   
 $y2 = 2$

**Call Level 2 1 4**

$n = \max \text{ length of } x \text{ or } y = 1$   
 $\text{return } 1 * 4 = 4$

**Call Level 2 0 2**

$n = \max \text{ length of } x \text{ or } y = 1$   
 $\text{return } 0 * 2 = 0$

**Call Level 2 1 2**

$n = \max \text{ length of } x \text{ or } y = 1$   
 $\text{return } 1 * 2 = 2$

$z = 4 + 0 - 2 = 2$

$\text{return } (4)*(10**(2)) + (2)*(10**1) + (0) = 420$

**Call Level 1 42 0**

$n = \max \text{ length of } x \text{ or } y = 2$   
 $m = n/2 = 1$   
 $x1 = 4$   
 $x2 = 2$   
 $y1 = 0$   
 $y2 = 0$

**Call Level 2 4 0**

$n = \max \text{ length of } x \text{ or } y = 1$   
 $\text{return } 4 * 0 = 0$

**Call Level 2 2 0**

$n = \max \text{ length of } x \text{ or } y = 1$   
 $\text{return } 2 * 0 = 0$

**Call Level 2 2 0**

$n = \max \text{ length of } x \text{ or } y = 1$   
 $\text{return } 2 * 0 = 0$

$$z = 0 + 0 - 0 = 0$$

$$\text{return } (0) * (10 ** (2)) + (0) * (10 ** 1) + (0) = 0$$

**Call Level 1 -32 42**

$$n = \text{max length of } x \text{ or } y = 2$$

$$m = n/2 = 1$$

$$x1 = -4$$

$$x2 = 8$$

$$y1 = 4$$

$$y2 = 2$$

**Call Level 2 -4 4**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } -4 * 4 = -16$$

**Call Level 2 8 2**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } 8 * 2 = 16$$

**Call Level 2 -12 2**

$$n = \text{max length of } x \text{ or } y = 2$$

$$m = n/2 = 1$$

$$x1 = -2$$

$$x2 = 8$$

$$y1 = 0$$

$$y2 = 2$$

**Call Level 3 -2 0**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } -2 * 0 = 0$$

**Call Level 3 8 2**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } 8 * 2 = 16$$

**Call Level 3 -10 -2**

$$n = \text{max length of } x \text{ or } y = 2$$

$$m = n/2 = 1$$

$$x1 = -1$$

$$x2 = 0$$

$$y1 = -1$$

$$y2 = 8$$

**Call Level 4 -1 -1**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } -1 * -1 = 1$$

**Call Level 4 0 8**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } 0 * 8 = 0$$

**Call Level 4 -1 -9**

$$n = \text{max length of } x \text{ or } y = 1$$

$$\text{return } -1 * -9 = 9$$

$$z = 1 + 0 - 9 = -8$$

```
    return 1*(10**(2)) + (-8)*(10**1) + 0 = 20
z = 0 + 16 - 20 = -4
    return (0)*(10**(2)) + (-4)*(10**1) + (16) = -24
z = -16 + 16 - (-24) = 24
    return (-16)*(10**(2)) + (24)*(10**1) + (16) = -1344
z = 420 + (0) - (-1344) = 1764
    return (420)*(10**(4)) + (1764)*(10**2) + (0) = 4376400
```